Git Lab: Collaborative Workflow for DevSecOps

Feature Branching, Conflicts, Merge Requests and Commit Discipline

Duration: about 2 h **Environment:** Linux terminal, Git ≥ 2.34, Python 3, a GitLab account

1 Your mission

In a DevSecOps pipeline, every test and scan is triggered by a Git event. If you push straight to `main`, resolve conflicts carelessly, merge without review or write vague commit messages, nobody can trust the pipeline or audit what happened. In this lab you work in a pair (one of you is the **Owner**, the other the **Partner**) and practice four habits:

#	Habit	Part	Why it matters
1	Feature branching	1	`main` stays safe; CI can test each branch separately
2	Local conflict resolution	2	Shared pipelines are never broken by a messy merge
3	Merge Request (MR)	3	Humans review the code while CI validates tests in the background
4	Commit discipline	4	Every commit linked to a ticket gives traceability and auditability

From your first commit, write messages as `type(scope): summary (#issue)`, for example `fix(greet): reject empty names (#2)`. Part 4 explains and enforces this.

2 Setup (10 min)

The Owner creates an empty GitLab project `devsecops-lab`, adds the Partner as *Developer*, and creates two issues: `#1` "Add punctuation to the greeting" and `#2` "Validate the name before greeting". Working alone? Use two clones in two folders to play both roles.

Owner, create and push the starter project:

```
mkdir devsecops-lab && cd devsecops-lab && git init
printf "__pycache__/\n.env\n" > .gitignore
printf 'def greet(name):\n    return "Hello, " + name\n' > app.py
cat > test_app.py <<'EOF'
import unittest
from app import greet

class TestGreet(unittest.TestCase):
    def test_greet(self):
        self.assertEqual(greet("Ana"), "Hello, Ana")

if __name__ == "__main__":
    unittest.main()
EOF
cat > .gitlab-ci.yml <<'EOF'
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
unit-tests:
  image: python:3.12-slim
  script: python -m unittest -v
EOF
git add . && git commit -m "chore: initial project (#0)"
git remote add origin git@gitlab.com:<owner>/devsecops-lab.git
git push -u origin main
```

Then protect `main`: *Settings* → *Repository* → *Protected branches*, push = **No one**, merge = **Developers + Maintainers**. **Partner**, clone the project with `git clone`. Both of you set your identity with `git config --global user.name` and `user.email`.

3 Part 1: Feature branching (20 min)

The rule: nobody commits on `main`. One issue = one branch = one MR. The pipeline then tests each branch in isolation and `main` always stays deployable. Name branches `feature/<issue>-<topic>` or `fix/<issue>-<`

topic>.

1.1 Try to break the rule

On `main`, commit anything and run `git push origin main`. GitLab rejects it. Undo with `git reset --hard origin/main`.

1.2 Add a local safety net

Hooks in `.git/hooks` are not shared, so keep them in a versioned folder:

```
mkdir .githooks
cat > .githooks/pre-push <<'EOF'
#!/bin/sh
while read local_ref local_sha remote_ref remote_sha; do
  [ "$remote_ref" = "refs/heads/main" ] && { echo "Push to main forbidden. Open an MR."; exit 1; }
done
exit 0
EOF
chmod +x .githooks/pre-push && git config core.hooksPath .githooks
```

Repeat 1.1: you are now stopped before the push leaves your machine.

1.3 One branch per issue

Both of you start from an up-to-date `main` (`git switch main && git pull`).

Owner (issue #1): create `feature/1-greeting-punctuation`, make `greet` return `"Hello, " + name + "!"`, update the test to expect `"Hello, Ana!"`, run `python3 -m unittest`, then:

```
git switch -c feature/1-greeting-punctuation
git commit -am "feat(greet): add exclamation mark (#1)"
git push -u origin feature/1-greeting-punctuation
```

Partner (issue #2), *before the Owner merges anything*: create `fix/2-validate-name` and change `greet` to:

```
def greet(name):
    if not name.strip():
        raise ValueError("name must not be empty")
    return "Hello, " + name.strip()
```

Add these two tests under `test_greet`, run the tests, then commit and push:

```
    def test_strip(self):
        self.assertEqual(greet("  Leo "), "Hello, Leo")
    def test_empty(self):
        with self.assertRaises(ValueError):
            greet("   ")
```

```
git switch -c fix/2-validate-name
git commit -am "fix(greet): reject empty names and trim input (#2)"
git push -u origin fix/2-validate-name
```

Q1. The server already blocks pushes to `main`. Why add a local hook too?

4 Part 2: Conflict resolution (30 min)

Two branches changed the same lines, and Git cannot choose for you. **Resolve conflicts locally, on your own branch, before the merge**, so the shared pipeline only ever sees clean code.

2.1 Create the conflict

Owner: open an MR for your branch, wait for the green pipeline, and merge it (Part 3 covers MRs in depth). **Partner**: open an MR for your branch. GitLab now says **"There are merge conflicts"**, because you both changed the `return` line.

2.2 Resolve it locally (Partner)

```
git fetch origin
git switch fix/2-validate-name
git merge origin/main
git diff --name-only --diff-filter=U     # conflicted files
```

Open `app.py`. Between `<<<<<<<` and `=======` is **your** version, between `=======` and `>>>>>>>` is the one from `main`:

```
<<<<<<< HEAD
    return "Hello, " + name.strip()
=======
    return "Hello, " + name + "!"
>>>>>>> origin/main
```

Keep *both* intentions (`return "Hello, " + name.strip() + "!"`) and delete every marker. Do the same in `test_app.py` if it conflicts. **Run the tests before committing**:

```
python3 -m unittest -v
```

`test_strip` fails: it still expects `"Hello, Leo"` without the `!`. The text merge worked, but the logic did not, so a resolved conflict is only correct once the tests pass. Fix the expected value, then:

```
git add app.py test_app.py
git commit -m "fix(greet): merge main and align tests (#2)"
git push
```

The MR is clean again and a new pipeline starts. Lost in the middle? `git merge --abort` takes you back to the start.

Good habits: merge `main` into your branch often so conflicts stay small; never accept "ours" or "theirs" for a whole file without reading it; always re-run the tests; talk to your partner if the conflict touches their logic.

Q2. Why is resolving a conflict on your own branch safer than on `main`?

5 Part 3: The Merge Request (30 min)

The MR is where code is accepted into `main`. Two checks work together: **humans** review design, logic and security (things nobody wrote a test for), while **CI** validates tests automatically in the background (fast, repeatable, but only checks what it was told to check). You need both.

3.1 Set up the gates (Owner)

In *Settings* → *Merge requests*, enable **Pipelines must succeed**, **All threads must be resolved** and **Delete source branch**. If your GitLab tier allows it, require 1 approval and forbid self-approval; otherwise follow the team rule: *nobody merges their own MR*. In the MR description, write `Closes #2`: it links the MR to the issue and closes it on merge.

3.2 Review in action

Partner, push a "forgotten" debug line on your branch:

```
    print("DEBUG user=" + name + " api_key=sk-test-1234567890")
```

```
git commit -am "chore(greet): add debug output (#2)" && git push
```

The pipeline stays **green**: no test looks for secrets. **Owner**, review the diff by eye using this checklist: Does it solve issue #2? Any hard-coded secret, unvalidated input or sensitive data in logs? Are the changes tested? Is it readable, with no leftovers? Do commits reference the issue? Comment on the `print` line to open a thread. **Partner**, remove the line, push, and resolve the thread.

3.3 CI blocks a bad change

Partner, break a test on purpose and push. The pipeline turns red and the **Merge** button is disabled. Fix the test, wait for green, let the Owner approve and merge.

Q3. In 3.2, which control caught the problem, human or CI? What could you add to the pipeline to catch it automatically next time?

6 Part 4: Commit message discipline (20 min)

A commit message is an audit record. Months later someone must answer: *who changed this, why, and under which ticket?* "update" or "fix stuff" makes that impossible, while a reference to the issue connects the code to the request, the review and the pipeline run.

Convention: `type(scope): imperative summary (#issue)` with type in `feat`, `fix`, `docs`, `chore`, `refactor`, `test`, `ci`, `security`; summary short (about 50 characters), no final period; add `Closes #N` in the body to auto-close the issue.

4.1 Rewrite

Rewrite these as compliant messages: `update`, `fix stuff`, `final version 2`, `removed password`.

4.2 Enforce it locally

```
cat > .githooks/commit-msg <<'EOF'
#!/bin/sh
msg=$(head -n1 "$1")
case "$msg" in Merge\ *) exit 0 ;; esac
if ! echo "$msg" | grep -Eq '^(feat|fix|docs|chore|refactor|test|ci|security)(\([a-z0-9-]+\))?: .{3,60}
     \(#[0-9]+\)$'; then
  echo "Invalid message. Expected: type(scope): summary (#issue)"; exit 1
fi
EOF
chmod +x .githooks/commit-msg
```

Try `git commit --allow-empty -m "fix stuff"`: refused. Then commit `.githooks` through a branch and MR (`ci(hooks): share git hooks (#3)`) so the whole team uses them.

4.3 Enforce it in the pipeline

Hooks can be skipped with `--no-verify`, so the server must check too. Add this job to `.gitlab-ci.yml`:

```
commit-lint:
  image: python:3.12
  variables: { GIT_DEPTH: "0" }
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
  script:
    - |
      bad=$(git log --no-merges --format=%s "$CI_MERGE_REQUEST_DIFF_BASE_SHA..HEAD" | grep -Ev '^(feat|fix|docs|
    chore|refactor|test|ci|security)(\([a-z0-9-]+\))?: .+ \(#[0-9]+|)$' || true)
      [ -z "$bad" ] || { echo "Non-compliant:"; echo "$bad"; exit 1; }
```

Push a commit with the message `"oops"` (bypass your hook with `--no-verify`) and watch `commit-lint` fail in the MR.

Q4. Why is the local hook not enough on its own?

7 Wrap-up: follow the audit trail (10 min)

Take issue #2 and reconstruct its story without asking anyone: the issue, its commits (`git log --oneline --grep ="#2"`), the MR with its review thread and approver, the pipelines that ran, and the merge on `main`. **Q5.** Which of the four habits made this trail possible?

8 Deliverables and grading (20 pts)

Submit the project link and screenshots of: the rejected push to `main`; the MR conflict and your resolved merge; the review thread, the red pipeline and the final green one; a rejected commit message and the failing `commit-lint` job; plus your answers to Q1–Q5.

Feature branching, protected `main`, no direct commits	4
Conflict resolved locally and tests re-run	4
MR quality: `Closes #N`, review thread, CI gate shown	5
Compliant messages, hook and `commit-lint` working	4
Answers and audit-trail reconstruction	3

Remember: never commit on `main`; resolve conflicts on your branch and re-test; an MR needs a human review and a green pipeline; every commit says what changed and which ticket it belongs to.